

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Проектирование интерфейсов пользователя. Синхронизация элементов
управления

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В.Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ.....	3
2 ЗАДАНИЕ	4
3 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ	5
4 ИСХОДНЫЙ КОД ПРОГРАММЫ	6
5 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ	20
6 ВЫВОД.....	21

1 Цель работы

Изучить способы синхронизации пользовательского интерфейса с внутренней базой данных приложения.

2 Задание

При разработке интерфейса, исследующего синхронизацию данных, обязательно использовать свойства класса с различными типами данных, а, следовательно, отобразить их значения в форме с помощью разнообразных элементов: `TextBox`, `CheckBox`, `RadioButton`, `ListBox`. Для тестирования разработанных методов обязательно использовать стандартные диалоговые окна

3 Описание работы программы

Основной функционал включает добавление, удаление и управление автомобилями, а также взаимодействие с их компонентами (двигатель, колеса). Взаимодействие с пользователем осуществляется через графический интерфейс, использующий элементы управления, такие как TextBox (для ввода характеристик автомобиля), CheckBox (для выбора ускоренного режима передвижения), RadioButton (для переключения режимов отображения информации), ListView (для отображения списка добавленных автомобилей). Каждый автомобиль представлен объектом класса Car, который имеет свойства с различными типами данных, включая string, int. Для отображения этих свойств на форме используются соответствующие элементы управления. Например, поле ввода TextBox позволяет задавать имя автомобиля, мощность двигателя и расход топлива. Для тестирования работы методов программы используются стандартные диалоговые окна. При успешном добавлении автомобиля в автопарк или запуске двигателя отображается сообщение через MessageBox. В случае ошибок (например, попытка передвижения с недостаточным количеством топлива) пользователю выводится соответствующее уведомление. Программа также поддерживает сохранение характеристик автомобилей в текстовый файл, если некоторые данные отсутствуют или не были введены. Это позволяет синхронизировать данные и сохранять их для последующего использования. Таким образом, интерфейс приложения обеспечивает удобное взаимодействие с пользователем, демонстрируя свойства класса и функционал через разнообразные элементы управления.

4 Исходный код программы

Ниже представлен исходный код программы.

Файл Car.cs:

```
using System;
using System.Linq;

namespace CarManagementApp
{
    // Класс Car представляет автомобиль с его компонентами (двигатель, колеса) и управлением ими
    public class Car
    {
        // Свойство Name хранит название автомобиля с обработкой лишних пробелов
        private string _name;
        public string Name
        {
            get => _name;
            set => _name = value?.Trim() ?? throw new ArgumentException("Название не может быть пустым.");
        }

        // Свойство Engine хранит объект двигателя автомобиля
        public Engine Engine { get; set; }

        // Свойство Wheels хранит массив колес автомобиля (массив из 4 колес)
        public Wheel[] Wheels { get; set; }

        // Свойство Fuel хранит текущее количество топлива в автомобиле с проверкой на отрицательные значения
        private int _fuel;
        public int Fuel
        {
            get => _fuel;
            set
            {
                if (value < 0)
                    throw new ArgumentException("Количество топлива не может быть отрицательным.");
                _fuel = value;
            }
        }

        // Свойство FuelTankCapacity хранит максимальный объем топливного бака
        public int FuelTankCapacity { get; private set; }

        // Свойство Mileage хранит пробег автомобиля в километрах
        public int Mileage { get; private set; } = 0;

        // Свойство FuelConsumption хранит расход топлива на 100 км
        public int FuelConsumption { get; private set; }

        // Статическое свойство для общего количества колес
        public static int WheelCount { get; } = 4;

        // Свойство Symbol проверяет регистр символа и преобразует строчные буквы в прописные
        private char _symbol;
        public char Symbol
        {
            get => _symbol;
            set => _symbol = char.IsLower(value) ? char.ToUpper(value) : value;
        }
    }
}
```

```

// Свойство LastServiceDate хранит дату последнего обслуживания с проверкой на корректность
private DateTime _lastServiceDate;
public DateTime LastServiceDate
{
    get => _lastServiceDate;
    set
    {
        if (value < DateTime.Now)
            throw new ArgumentException("Дата не может быть в прошлом.");
        _lastServiceDate = value;
    }
}

// Конструктор класса Car
// Инициализирует автомобиль с заданными параметрами: имя, мощность двигателя, расход топлива и
объем бака
public Car(string name, int horsepower, int fuelConsumption, int fuelTankCapacity)
{
    Name = name;

    Engine = new Engine { HorsePower = horsepower };
    Wheels = new Wheel[4]; // Инициализация массива колес

    // Инициализация колес и подписка на событие поломки каждого колеса
    for (int i = 0; i < WheelCount; i++)
    {
        Wheels[i] = new Wheel();
        Wheels[i].ComponentBroken += OnComponentBroken; // Подписка на событие поломки колеса
    }

    Engine.ComponentBroken += OnComponentBroken; // Подписка на событие поломки двигателя

    FuelConsumption = fuelConsumption;
    FuelTankCapacity = fuelTankCapacity;
    Fuel = FuelTankCapacity; // Заполнение бака до максимума
}

// Свойство IsEngineOn проверяет, работает ли двигатель
// Возвращает true, если двигатель включен, иначе false
public bool IsEngineOn => Engine.IsRunning;

// Метод StartEngine запускает двигатель
public void StartEngine() => Engine.Start();

// Метод StopEngine выключает двигатель
public void StopEngine() => Engine.Stop();

// Обработчик события поломки компонента (колеса или двигателя)
// При поломке компонента выводится сообщение о поломке в консоль
private void OnComponentBroken(object sender, EventArgs e)
{
    if (sender is Engine)
    {
        // Сообщение о поломке двигателя
        Console.WriteLine($"{Name}: Двигатель сломан!");
    }
    else if (sender is Wheel wheel)
    {
        // Сообщение о поломке колеса
    }
}

```

```

        Console.WriteLine($"{Name}: Колесо {Array.IndexOf(Wheels, wheel) + 1} сломано!");
    }
}

// Перегрузка метода Move: стандартное движение (обычная скорость)
// Этот метод вызывает второй метод Move с параметром isFast = false (стандартная скорость)
public void Move(int distance)
{
    Move(distance, false); // Перемещение с обычной скоростью
}

// Перегрузка метода Move: движение с дополнительными параметрами (например, высокая скорость)
// Если параметр isFast равен true, расход топлива снижается на 20% (ускоренный режим)
public void Move(int distance, bool isFast = false)
{
    // Проверка, включен ли двигатель, если выключен - нельзя двигаться
    if (!IsEngineOn)
    {
        throw new InvalidOperationException("Двигатель выключен! Невозможно передвигаться.");
    }

    // Проверка, не сломан ли двигатель
    if (Engine.Condition == "Сломано")
        throw new InvalidOperationException("Двигатель сломан, машина не может двигаться.");

    // Проверка, достаточно ли топлива для движения
    if (Fuel < (distance * FuelConsumption) / 100)
        throw new InvalidOperationException("Недостаточно топлива!");

    // Проверка, не сломано ли одно из колес
    if (Wheels.Any(w => w.Condition == "Сломано"))
        throw new InvalidOperationException("Одно из колес сломано!");

    // Увеличиваем пробег
    Mileage += distance;

    // Расчет расхода топлива в зависимости от скорости
    if (isFast)
    {
        // Ускоренная скорость: расход топлива снижается на 20%
        Fuel -= (int)((distance * FuelConsumption) / 100 * 0.8); // Приведение типа к int
    }
    else
    {
        // Обычная скорость: расход топлива без изменений
        Fuel -= (distance * FuelConsumption) / 100;
    }

    // Поломка колеса каждые 200 км
    if (Mileage % 200 == 0)
    {
        Random random = new Random();
        int brokenWheelIndex = random.Next(Wheels.Length);
        Wheels[brokenWheelIndex].ChangeCondition("Сломано"); // Случайная поломка одного из колес
    }

    // Поломка двигателя каждые 1000 км
    if (Mileage % 1000 == 0)
    {
        Engine.ChangeCondition("Сломано"); // Поломка двигателя после 1000 км пробега
    }
}

```

```

// Метод Refuel заправляет автомобиль на указанное количество литров
// Проверяет, не превышает ли заправка емкость бака
public void Refuel(int amount)
{
    if (amount <= 0)
        throw new ArgumentException("Заправляемое количество должно быть положительным!");

    Fuel += amount;
    if (Fuel > FuelTankCapacity)
        Fuel = FuelTankCapacity; // Если заправлено больше, чем вмещает бак, устанавливаем максимальную
емкость
}

// Метод ReplaceBrokenWheel заменяет сломанное колесо на новое
public void ReplaceBrokenWheel()
{
    for (int i = 0; i < Wheels.Length; i++)
    {
        if (Wheels[i].Condition == "Сломано")
        {
            Wheels[i].ChangeCondition("Новое"); // Заменить сломанное колесо
            return;
        }
    }
    throw new InvalidOperationException("Нет сломанных колес для замены."); // Если нет сломанных колес
}

// Метод для получения информации о машине
// Если параметр showDetailed = true, выводится подробная информация
public virtual string GetCarInfo(bool showDetailed = false)
{
    // Основная информация о машине
    string info = $"Название: {Name}\n" +
        $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
        $"Пробег: {Mileage} км\n" +
        $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
        $"Расход топлива: {FuelConsumption} л/100 км\n" +
        $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
        $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

    // Если showDetailed == true, добавляется подробная информация
    if (showDetailed)
    {
        info += $"Оставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
        for (int i = 0; i < Wheels.Length; i++)
        {
            info += $"Колесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
        }
    }

    return info;
}
}
}

```

Файл Form1.cs (приложение):

```
using System;
```

```

using System.Windows.Forms;

using CarManagementApp;

namespace CarManagementApp
{
    public partial class Form1 : Form
    {
        // Объект автопарка, который будет хранить все автомобили

        private CarPark carPark = new CarPark();

        // Конструктор формы, инициализация компонентов

        public Form1()
        {
            InitializeComponent();

        }

        // Обработчик кнопки "Создать авто"

        // При нажатии создает новый объект Car с параметрами из текстовых полей, добавляет его в
автопарк

        private void buttonAddCar_Click(object sender, EventArgs e)
        {
            try
            {
                // Получаем данные из текстовых полей

                string name = textBoxCarName.Text;

                int horsepower = int.Parse(textBoxHorsePower.Text);

                int fuelConsumption = int.Parse(textBoxFuelConsumption.Text);

                int fuelTankCapacity = int.Parse(textBoxFuelTankCapacity.Text);

```

```

        // Создаем новый автомобиль

        var car = new Car(name, horsePower, fuelConsumption, fuelTankCapacity);

        // Добавляем автомобиль в автопарк

        carPark.AddCar(car);

        // Обновляем интерфейс: добавляем машину в список и выводим действия

        listBoxCars.Items.Add(name);

        listBoxActions.Items.Add($"Добавлен автомобиль: {name}");

    }

    catch (Exception ex)

    {

        // В случае ошибки выводим сообщение

        MessageBox.Show($"Ошибка при добавлении автомобиля: {ex.Message}");

    }

}

// Обработчик кнопки "Удалить авто"

// При нажатии удаляет выбранный автомобиль из автопарка

private void buttonRemoveCar_Click(object sender, EventArgs e)

{

    try

    {

        // Получаем выбранную машину из списка

        string selectedCar = listBoxCars.SelectedItem?.ToString();

        if (selectedCar != null)

        {

            // Удаляем машину из автопарка

            carPark.RemoveCar(selectedCar);

```

```

        // Обновляем интерфейс: удаляем машину из списка и выводим действия
        listBoxCars.Items.Remove(selectedCar);

        listBoxActions.Items.Add($"Удален автомобиль: {selectedCar}");
    }

    else

    {

        // Если машина не выбрана, выводим предупреждение
        MessageBox.Show("Выберите автомобиль для удаления.");
    }
}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение
    MessageBox.Show($"Ошибка при удалении автомобиля: {ex.Message}");
}
}

// Обработчик кнопки "Запустить двигатель"

// При нажатии запускает двигатель выбранного автомобиля
private void buttonStartEngine_Click(object sender, EventArgs e)
{
    try
    {
        // Получаем выбранную машину из списка
        string selectedCar = listBoxCars.SelectedItem?.ToString();

        if (selectedCar != null)
        {
            // Получаем объект автомобиля из автопарка и запускаем двигатель

```

```

        Car car = carPark.GetCar(selectedCar);

        car.StartEngine();

        // Обновляем интерфейс: выводим действия

        listBoxActions.Items.Add($"Двигатель автомобиля {selectedCar} запущен.");

    }

    else

    {

        // Если машина не выбрана, выводим предупреждение

        MessageBox.Show("Выберите автомобиль для запуска двигателя.");

    }

}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение

    MessageBox.Show($"Ошибка при запуске двигателя: {ex.Message}");

}

}

// Обработчик кнопки "Выключить двигатель"

// При нажатии выключает двигатель выбранного автомобиля

private void buttonStopEngine_Click(object sender, EventArgs e)

{

    try

    {

        // Получаем выбранную машину из списка

        string selectedCar = listBoxCars.SelectedItem?.ToString();

        if (selectedCar != null)

        {

```

```

        // Получаем объект автомобиля из автопарка и выключаем двигатель

        Car car = carPark.GetCar(selectedCar);

        car.StopEngine();

    }

    // Обновляем интерфейс: выводим действия

    listBoxActions.Items.Add($"Двигатель автомобиля {selectedCar} остановлен.");

}

else

{

    // Если машина не выбрана, выводим предупреждение

    MessageBox.Show("Выберите автомобиль для остановки двигателя.");

}

}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение

    MessageBox.Show($"Ошибка при остановке двигателя: {ex.Message}");

}

}

// Обработчик кнопки "Заправка"

// При нажатии заправляет выбранный автомобиль на указанное количество топлива

private void buttonRefuel_Click(object sender, EventArgs e)

{

    try

    {

        // Получаем выбранную машину из списка

        string selectedCar = listBoxCars.SelectedItem?.ToString();

        if (selectedCar != null)

```

```

{

    // Получаем количество топлива из текстового поля

    int fuelAmount = int.Parse(textBoxFuelAmount.Text);


    // Получаем объект автомобиля из автопарка и заправляем его

    Car car = carPark.GetCar(selectedCar);

    car.Refuel(fuelAmount);


    // Обновляем интерфейс: выводим действия

    listBoxActions.Items.Add($"Автомобиль {selectedCar} заправлен на {fuelAmount} литров.");

}

else

{

    // Если машина не выбрана, выводим предупреждение

    MessageBox.Show("Выберите автомобиль для заправки.");

}

}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение

    MessageBox.Show($"Ошибка при заправке автомобиля: {ex.Message}");

}

}

// Обработчик кнопки "Передвижение"

// При нажатии передвигает выбранный автомобиль на указанное расстояние

private void buttonMove_Click(object sender, EventArgs e)

{

    try

```

```

{

    // Получаем выбранную машину из списка

    string selectedCar = listBoxCars.SelectedItem?.ToString();

    if (selectedCar != null)

    {

        // Получаем расстояние из текстового поля

        int distance = int.Parse(textBoxDistance.Text);


        // Проверка, выбрал ли пользователь ускоренный режим движения

        bool isFast = checkBoxFastMode.Checked;


        // Вызов перегруженного метода Move с параметром для высокой скорости

        if (isFast)

        {

            // Перемещение с высокой скоростью

            Car car = carPark.GetCar(selectedCar);

            car.Move(distance, true); // Перегруженная версия метода Move с параметром isFast

            listBoxActions.Items.Add($"Автомобиль {selectedCar} проехал {distance} км с высокой
скоростью.");

        }

        else

        {

            // Стандартное перемещение

            Car car = carPark.GetCar(selectedCar);

            car.Move(distance); // Стандартная версия метода Move

            listBoxActions.Items.Add($"Автомобиль {selectedCar} проехал {distance} км.");

        }

    }

    else

```

```

    {

        // Если машина не выбрана, выводим предупреждение

        MessageBox.Show("Выберите автомобиль для передвижения.");

    }

}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение

    MessageBox.Show($"Ошибка при движении автомобиля: {ex.Message}");

}

}

// Обработчик кнопки "Заменить колесо"

// При нажатии заменяет сломанное колесо выбранного автомобиля

private void buttonReplaceWheel_Click(object sender, EventArgs e)

{

    try

    {

        // Получаем выбранную машину из списка

        string selectedCar = listBoxCars.SelectedItem?.ToString();

        if (selectedCar != null)

        {

            // Получаем объект автомобиля из автопарка и заменяем колесо

            Car car = carPark.GetCar(selectedCar);

            car.ReplaceBrokenWheel();

            // Обновляем интерфейс: выводим действия

            listBoxActions.Items.Add($"Колесо автомобиля {selectedCar} заменено.");

        }

    }

}

```

```

else

{

    // Если машина не выбрана, выводим предупреждение

    MessageBox.Show("Выберите автомобиль для замены колеса.");

}

}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение

    MessageBox.Show($"Ошибка при замене колеса: {ex.Message}");

}

}

// Обработчик кнопки "Показать информацию"

// При нажатии отображает информацию о выбранном автомобиле

private void buttonShowInfo_Click(object sender, EventArgs e)

{

    try

    {

        // Получаем выбранную машину из списка

        string selectedCar = listBoxCars.SelectedItem?.ToString();

        if (selectedCar != null)

        {

            // Получаем объект автомобиля из автопарка

            Car car = carPark.GetCar(selectedCar);

            bool showDetails = checkBoxShowDetails.Checked; // Получаем состояние чекбокса для
подробной информации

            // Получаем информацию о машине

```

```

выбран
        string info = car.GetCarInfo(showDetails); // Получаем подробную информацию, если чекбокс

        MessageBox.Show(info); // Показываем информацию о машине
    }

    else

    {

        // Если машина не выбрана, выводим предупреждение

        MessageBox.Show("Выберите автомобиль для отображения информации.");

    }

}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение

    MessageBox.Show($"Ошибка при отображении информации: {ex.Message}");

}

}

}

}

```

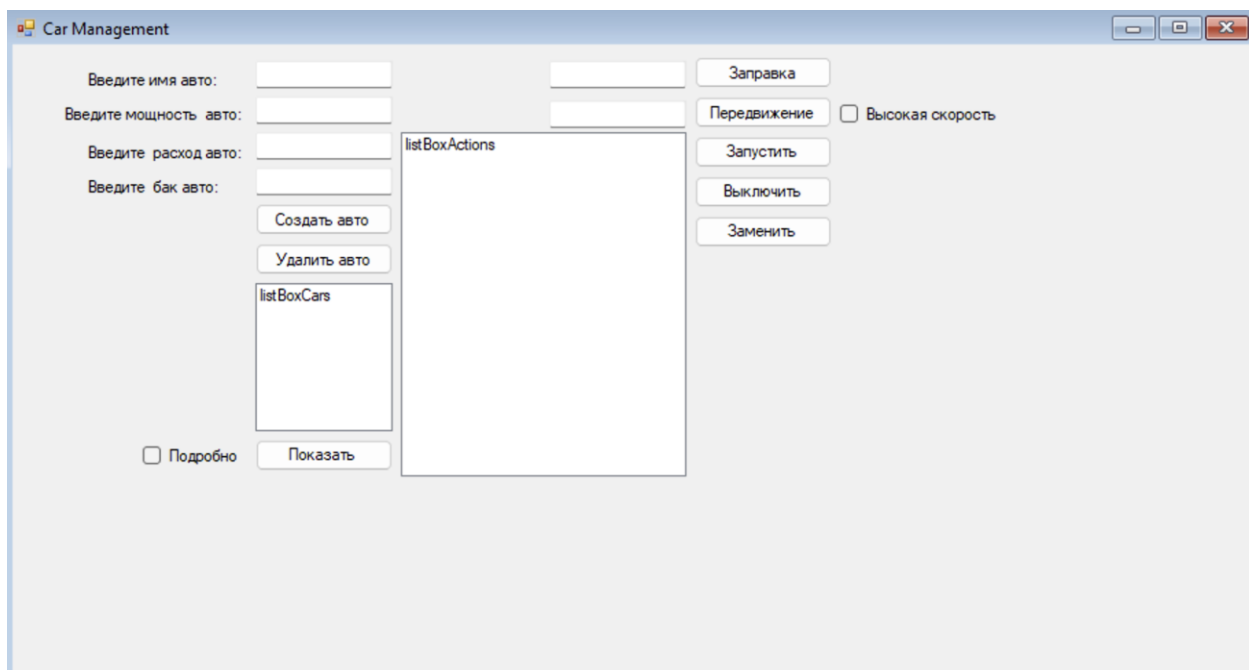


Рисунок 1 – Расположение графических элементов

5 Результат работы программы

Если нажать кнопку «Создать», но не ввести название авто или другие пункты, то отобразится ошибка (Рисунок 2).

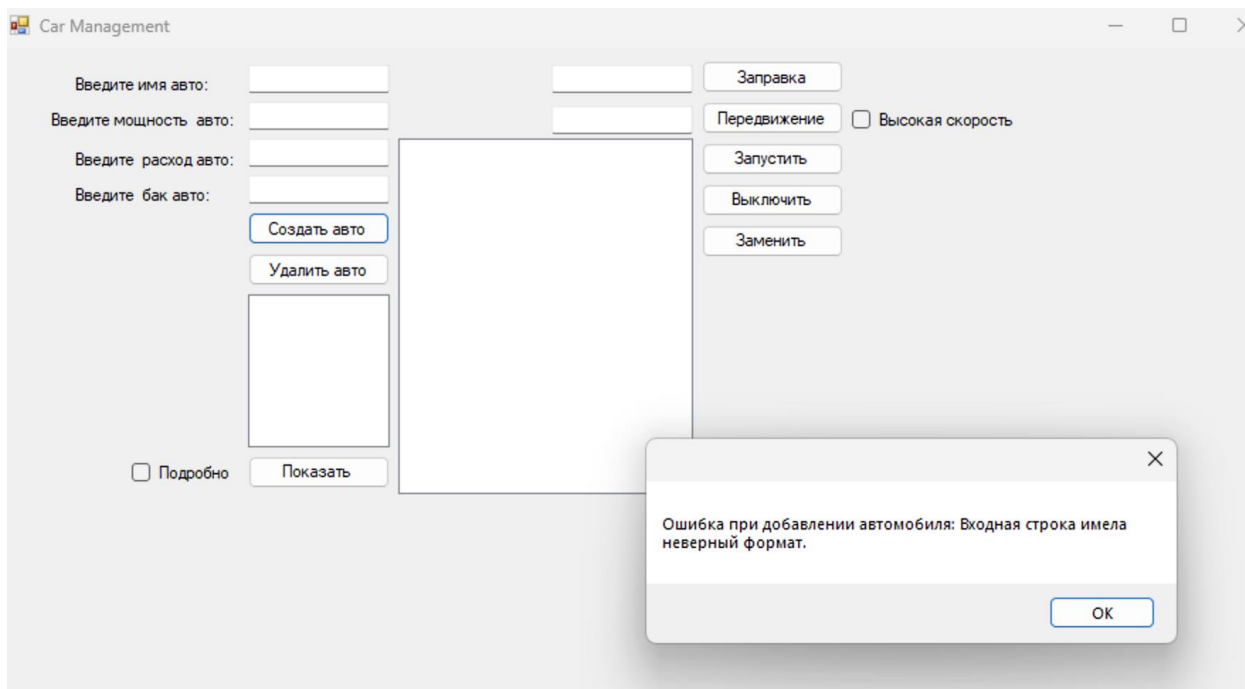


Рисунок 2 – Метод проверки ввода

Заполним все поля и добавим авто – она сохранится в списке в левой части интерфейса (Рисунок 3). При нажатии на нее отображается соответствующая ей информация.

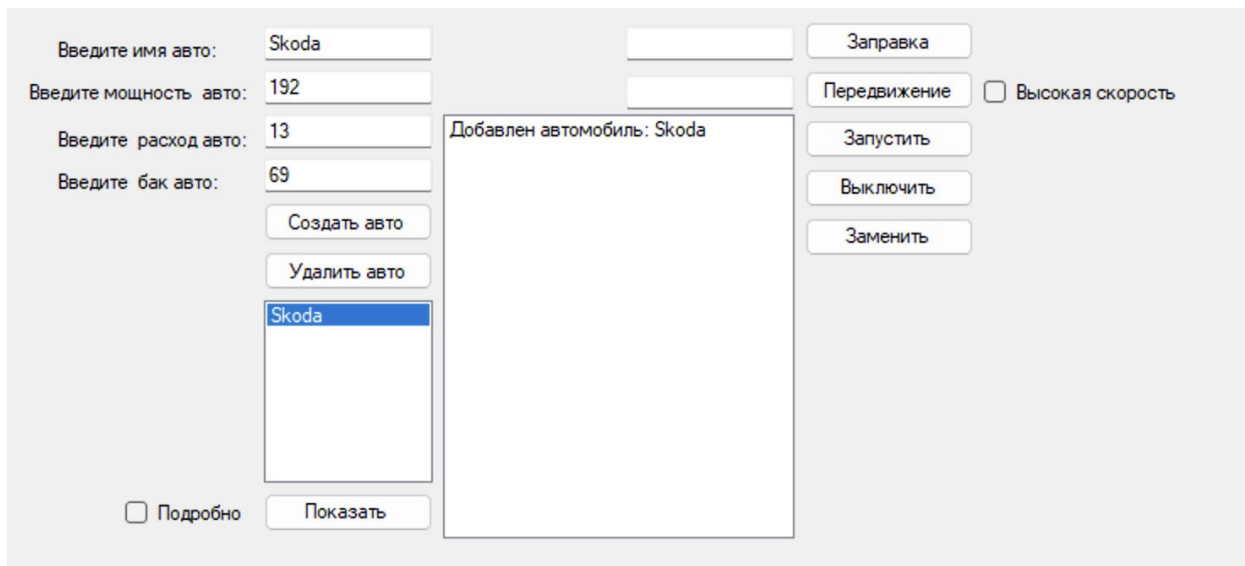


Рисунок 3 – Метод MessageBox

6 Вывод

В этой лабораторной работе мной были методы классов, с помощью них была осуществлена привязка выводимых данных к соответствующим полям вывода. Данные для вывода импортировались из выбранного элемента списка listBox.